

CSC 330: Assignment 3

Deadline: Sun, Mar 1 at 23.59

Submit via: [CSC 330 Submission Portal](#)

Ground rules

- You must complete this assignment *entirely* on your own. In other words, you should come up with the solution *yourself*, write the code *yourself*, and test the code *yourself*. The university policies on academic dishonesty (a.k.a. cheating) will be taken very seriously. Check the syllabus for details on what is allowed and what is not.
 - You can have high-level discussions with your classmates about the course material.
 - You are also welcome to use Teams or come to office hours and ask questions. If in doubt, please ask—we are here to help!
 - Using LLMs to get the solutions is akin to shooting yourself in the foot: you *might* pass the assignment, but you will most likely fail the written exams.
- You can use a maximum of **three (3) late days per assignment**, and a maximum of **seven (7) days for the whole semester**. Any further accommodations must be announced and negotiated with the instructor at least seven (7) days before the deadline.
- You will need to gather at least half of the total assignment score to pass the course.
- If you think some things should be explained in detail, please include the explanations in the form of source code comments.
- We will try to grade your assignments within seven (7) days of the initial submission deadline.
- If you think there is a problem with your grade, you have one week to raise concerns after the grades go public. TAs will be holding office hours during those seven days to address any such problems. After that, your grade is set in stone.
- Document your code where needed!
- Watch your style! We will penalize submissions with inadequate or bad style.
- Your programs should generate **no** errors or warnings. If a program generates a single compilation error or warning, it will receive a zero.
- Please indicate your V-number and name within a comment at the top of each submitted file.
 - **Do not** include other private information (full name, your address, telephone number, medical exam tests etc.).

Furthermore, for this assignment:

- Use the provided templates and tests in the attached `code.zip`. Replace `TODD` parts with your code. Leave the rest alone.
- You should only submit a single ZIP file containing two Racket files in `.rkt` format to the CSC 330 Submission Portal. The contents of the ZIP must include `prob1.rkt` and `prob2.rkt` (lowercase).
 - Warning: different file names might result in zero points. Our scripts expect the correct file names.
 - Another warning: put the filenames at the root of the ZIP file; if you put them within a directory, our script might not be able to find it.
- You cannot use mutation. Use of any function that ends with `!` is forbidden.
- Try not to stray away from the features discussed in class.

Test examples, updates and error corrections will be available on Teams and the CSC 330 Submission Portal. Make sure to check it from time to time.

Testing

Use the provided files (`prob1.rkt` and `prob2_test.rkt`) for sample test cases. I recommend you start by commenting all but the first one. Move one test group at a time. It will make your debugging faster.

The tests are organized in such a way that they will help you complete your assignment.

Problem 1: Warm-up [8 points]

1. [2 points] Write a function `stream-for-n-steps` that takes a stream `s` and a number `n`. It returns a list holding the first n values produced by `s` in order.
 - **Hint:** Use the provided `nat-num-stream` to test this function.
2. [2 points] Write a stream called `fibo-stream` that corresponds to the Fibonacci sequence.
 - **Hint:** Use `stream-for-n-steps` to test this function.
3. [2 points] Write a function called `filter-stream` that takes two parameters (the first is a function, the second a stream) and returns a stream. The function returns a boolean that determines if a given value returned by the stream is to be kept or not.
 - **Hint:** `filter-stream` is similar to `filter`, but instead of a list, it is filtering a stream.
4. [2 points] There exists a family of numeric sequences that follow a simple pattern:

$$f(i_0), f(i_1), f(i_2), f(i_3), \dots$$

where

$$i_j = i_{j-1} + \delta.$$

Define a macro called `create-stream` that will create such streams. It will take 3 parameters: `f`, `i0` and `delta`. The integer `i0` corresponds to i_0 . The syntax of the macro should be:

(create-stream name using `f` starting at `i0` with increment `delta`)

This will create a stream called `name` using `f` as its function, with starting value `i0` and increment `delta`.

- **Note 1:** `i0` should be evaluated only once when the stream is first used to generate the first value.
- **Note 2:** `delta` should be evaluated once every time the stream is used (except in the first invocation).

Problem 2: MF Language [42 points]

This is probably the most important assignment that you will do in this course!

This assignment has to do with MF/PL (a *My First Programming Language*). MF/PL programs are written directly in Racket by using the constructors defined by the structs defined at the beginning of `prob2.rkt`. This is the definition of MF/PL's syntax:

1. If `s` is a Racket string, then `(var s)` is a MF/PL expression (variable use).
2. If `n` is a Racket integer, then `(int n)` is a MF/PL expression (constant).
3. If `e1` and `e2` are MF/PL expressions, then `(add e1 e2)` is a MF/PL expression (addition).
4. If `s1` and `s2` are Racket strings and `e` is a MF/PL expression, then `(fun s1 s2 e)` is a MF/PL expression (function). In `e`, `s1` is bound to the function itself (to allow recursion), and `s2` is bound to the (single) argument.
 - **Special case:** `(fun #f s2 e)` is allowed for anonymous nonrecursive functions.
5. If `e1`, `e2`, and `e3`, and `e4` are MF/PL expressions, then `(if> e1 e2 e3 e4)` is a MF/PL expression. It is a conditional where the result is `e3` if `e1` is strictly greater than `e2`, otherwise the result is `e4`. Only one of `e3` and `e4` is evaluated.
6. If `e1` and `e2` are MF/PL expressions, then `(call e1 e2)` is a MF/PL expression (function call).
7. If `s` is a Racket string and `e1` and `e2` are MF/PL expressions, then `(mlet s e1 e2)` is a MF/PL expression (a let expression where the value resulting `e1` is bound to `s` in the evaluation of `e2`).
8. If `e1` and `e2` are MF/PL expressions, then `(apair e1 e2)` is a MF/PL expression (pair-creator).
9. If `e1` is a MF/PL expression, then `(fst e1)` is a MF/PL expression (getting the first part of a pair).
10. If `e1` is a MF/PL expression, then `(snd e1)` is a MF/PL expression (getting the second part of a pair).
11. `(aunit)` is a MF/PL expression that holds no data (much like `()` in OCaml or `null` in Racket).
 - **Note:** `(aunit)` is a MF/PL expression, but `aunit` is not.
12. If `e1` is a MF/PL expression, then `(isaunit e1)` is a MF/PL expression (testing for `(aunit)`).
13. `(closure env f)` is a MF/PL value where `f` is MF/PL function (an expression made from `fun`) and `env` is an environment mapping variables to values.
 - **Important:** Closures do not appear in source programs; they result from evaluating functions.

A MF/PL value is either a MF/PL integer constant, closure, `aunit`, or pair of (MF/PL) values. Similar to Racket, we can build list values out of nested pair values that end with a MF/PL `aunit`. Such a MF/PL value is called a MF/PL list. You should assume MF/PL programs are syntactically correct: do not worry about wrong things like `(int "hi")` or `(int (int 37))`. However, do not assume MF/PL programs are free of type errors like `(add (aunit) (int 7))` or `(fst (int 7))`.

Warnings:

- What makes this assignment challenging is that **you have to understand MF/PL well** and debugging an interpreter is an acquired skill.
- **Do not use any mutation** (`set!`, `set-mcar!`, etc.) anywhere in the assignment.

Getting started! [4 points]

1. [2 points] Write a Racket function `rkt-list→mfpl-list` that takes a Racket list (presumably of MF/PL values but that will not affect your solution) and produces an analogous MF/PL list with the same elements in the same order.
 2. [2 points] Write a Racket function `mfpl-list→rkt-list` that takes a MF/PL list (presumably of MF/PL values but that will not affect your solution) and produces an analogous Racket list (of MF/PL values) with the same elements in the same order.
- **Hint:** look at the test cases to see what a MF/PL list looks like and what its equivalent Racket list is.

Implementing the MF/PL Language [20 points]

3. [20 points] Write a MF/PL interpreter: a Racket function `eval-exp` that takes a MF/PL expression `e` and either returns the MF/PL value that `e` evaluates to under the empty environment. If evaluation encounters a run-time MF/PL type error or an unbound MF/PL variable, the function should raise a Racket error. A MF/PL expression is *always* evaluated under an environment (for evaluating variables, as usual). In your interpreter, use a Racket list of Racket pairs to represent this environment (which is initially empty) so that you can use without modification the provided `envlookup` function.

Here is a description of the semantics of MF/PL expressions:

- All values (including closures) evaluate to themselves. For example, `(eval-exp (int 17))` would return `(int 17)`, not `17`.
- A variable evaluates to the value associated with it in the environment.

- An addition evaluates its subexpressions and assuming they both produce integers, produces the integer that is their sum.
 - **Note:** This case is done for you to get you pointed in the right direction.
- Functions are lexically scoped: A function evaluates to a closure holding the function and the current environment.
- An `if>` evaluates its first two subexpressions to values `v1` and `v2`, respectively. If both values are integers, it evaluates its third subexpression if `v1` is a strictly greater integer than `v2`, otherwise it evaluates its fourth subexpression.
- An `mlet` expression evaluates its first expression to a value `v`. Then it evaluates the second expression to a value, in an environment extended to map the name in the `mlet` expression to `v`. **It must be placed at the beginning of the current environment.**
- A call evaluates its first and second subexpressions to values. If the first is not a closure, it is an error. Else, it evaluates the closure's function's body in the closure's environment extended to map the function's name to the closure (unless the name field is `#f`) and the function's argument to the result of the second subexpression (in that order).
 - **Remember:** it is a closure, hence, the evaluation of the function is in the environment where the function is defined, not the current environment (extended with the function name and its parameters—which should be placed at the beginning of its environment).
- A `apair` expression evaluates its two subexpressions and produces a (new) pair holding the results.
- A `fst` expression evaluates its subexpression. If the result for the subexpression is a pair, then the result for the `fst` expression is the `e1` field in the pair.
- A `snd` expression evaluates its subexpression. If the result for the subexpression is a pair, then the result for the `snd` expression is the `e2` field in the pair.
- An `isaunit` expression evaluates its subexpression. If the result is an `aunit` expression, then the result for the `isaunit` expression is the MF/PL integer 1, else the result is the MF/PL integer 0.

Hints:

- **Hint 1:** See the test cases to get a better understanding of what you have to create.
- **Hint 2:** Inspect the implementation of `add` and `var`, which are provided (inside `eval-under-env`). Do not start the code until you understand how they work.
- **Hint 3:** The `call` case is the most complicated. In my solution, no case is more than 15 lines and several are 1 line.

Expanding the Language [10 points]

MF/PL is a small language, but we can write Racket functions that act like MF/PL macros so that users of these functions feel like MF/PL is larger. The Racket functions produce MF/PL expressions that could then be put inside larger MF/PL expressions or passed to `eval-exp`. In implementing these Racket functions, do not use `closure` (which is used only internally in `eval-exp`). Also do not use `eval-exp` (we are creating a program, not running it).

- [2 points] Write a Racket function `ifaunit` that takes three MF/PL expressions `e1`, `e2`, and `e3`. It returns a MF/PL expression that when run evaluates `e1` and if the result is MF/PL's `aunit` then it evaluates `e2` and that is the overall result, else it evaluates `e3` and that is the overall result. Sample solution: 1 line.
- [4 points] Write a Racket function `mlet*` that takes a Racket list of Racket pairs `'((s_1 . e_1) ... (s_i . e_i) ... (s_N . e_N))` and a final MF/PL expression `e`. In each pair, assume `s_i` is a Racket string and `e_i` is a MF/PL expression. `mlet*` returns a MF/PL expression whose value is `e` evaluated in an environment where each `s_i` is a variable bound to the result of evaluating the corresponding `e_i` for $1 \leq i \leq N$. The bindings are done sequentially, so that each `e_i` is evaluated in an environment where `s_1` through `s_j` have been previously bound to the values `e_1` through `e_j` where $j < i$.
- [4 points] Write a Racket function `if=` that takes four MF/PL expressions `e1`, `e2`, `e3`, and `e4` and returns a MF/PL expression that acts like `if>` except `e3` is evaluated if and only if `e1` and `e2` are equal integers. Assume none of the arguments to `if=` use the MF/PL variables (e.g., `x` or `y`). Use this assumption so that when an expression returned from `if=` is evaluated, `e1` and `e2` are each evaluated exactly once.
 - **Hint:** You have to figure out how to implement `if=` using `if>`.

Using the language [8 points]

We can write MF/PL expressions directly in Racket using the constructors for the structs and (for convenience) the functions we wrote in the previous problem.

- [4 points] Bind to the Racket variable `mfpl-map` a MF/PL function that acts like `map` (as we used extensively in OCaml). Your function should be curried: it should take a MF/PL function and return a MF/PL function that takes a MF/PL list and applies the function to every element of the list returning a new MF/PL list. Recall a MF/PL list is `aunit` or a pair where the second component is a MF/PL list.

8. [4 points] Bind to the Racket variable `mfp1-map-add-N` a MF/PL function that takes an MF/PL integer `i` and returns a MF/PL function that takes a MF/PL list of MF/PL integers and returns a new MF/PL list of MF/PL integers that adds `i` to every element of the list. Use `mfp1-map` (a use of `mlet` is given to you to make this easier).

Good luck!